



Collaborative Filtering

2026 A&I 4th

구승민

목차

1. 코드 실습

- MovieLens 데이터셋을 통해 추천 모델 만들기

2. 코드 설명

- 실습에서 활용된 코드의 동작 이해하기

3. EDA, Item vs User

- 구현 방식에 따른 모델의 결과 확인하기

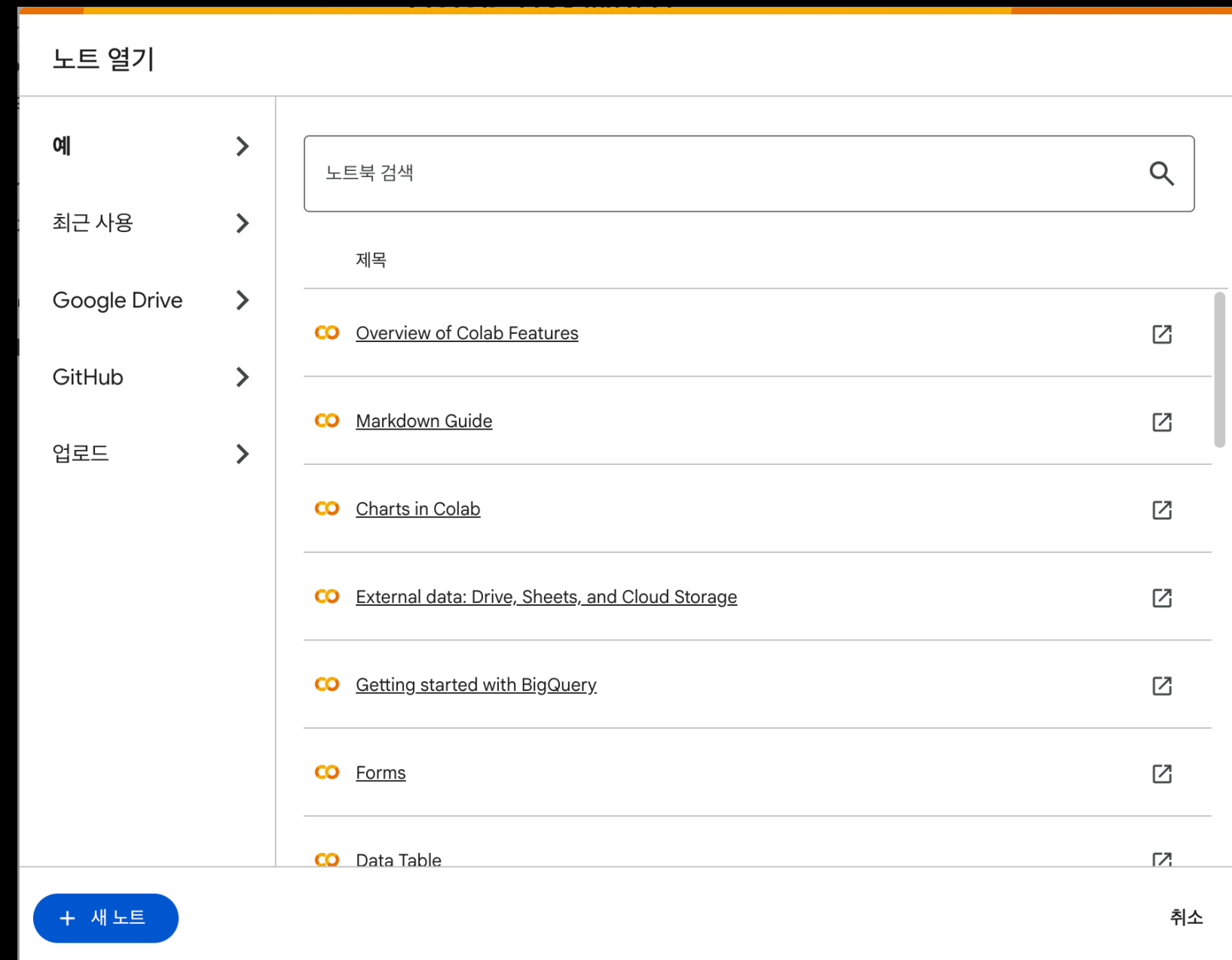
1. 코드 실습

- 그냥 따라하시면 됩니다

실습 환경 구성하기

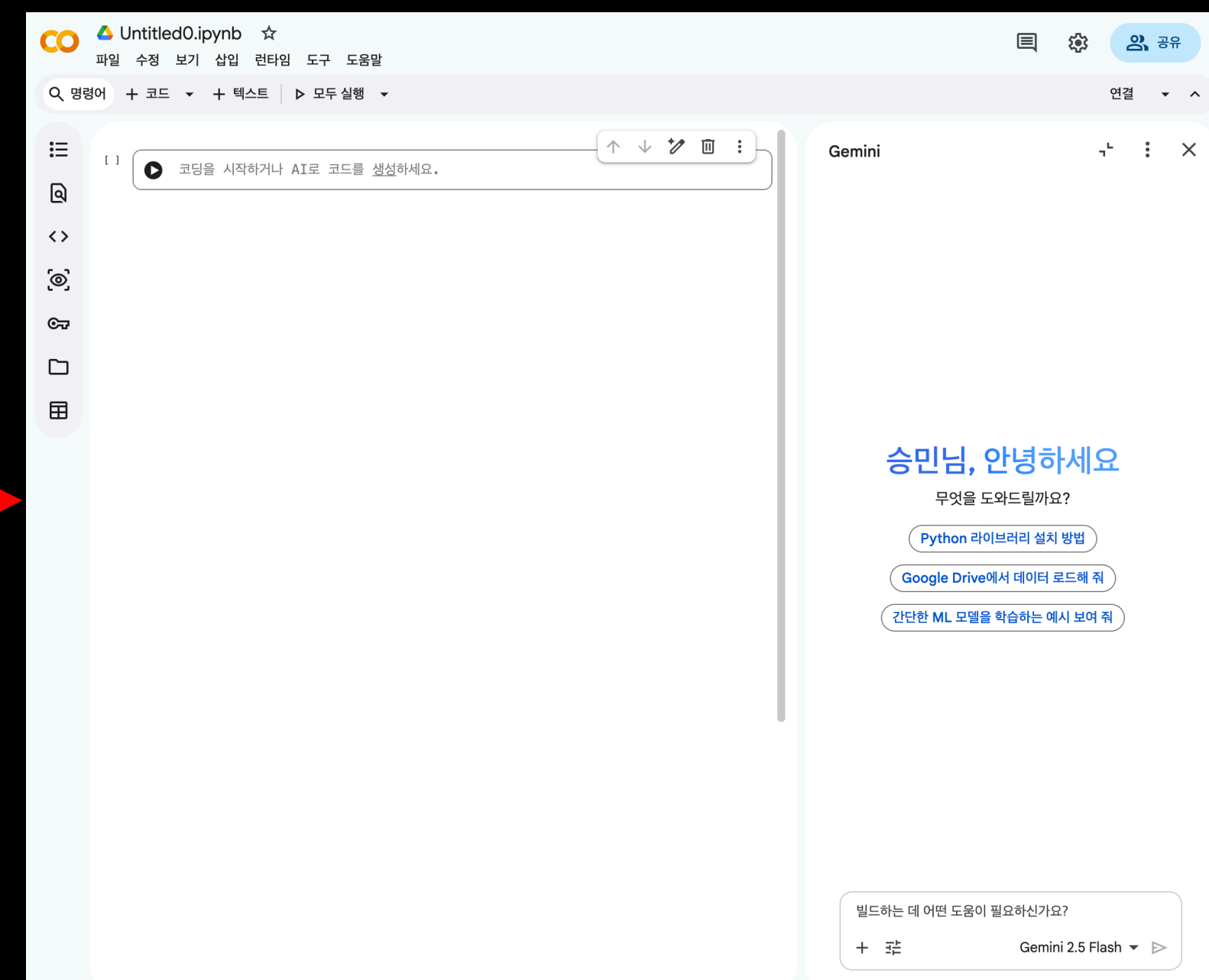
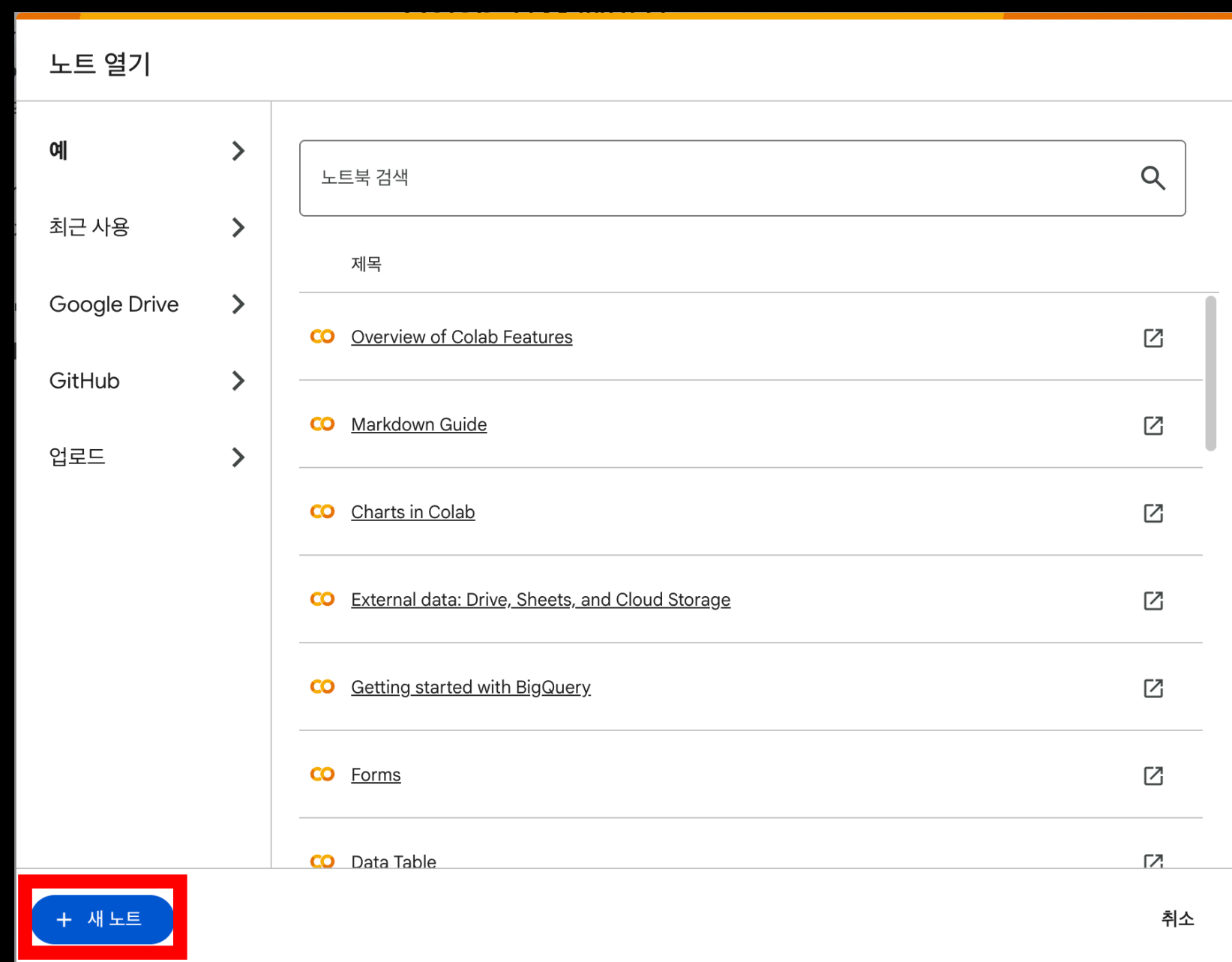
아래 링크로 이동

- <https://colab.research.google.com>



실습 환경 구성하기

하단 새 노트 클릭하여 코드 환경 구성



코랩 단축키

[실행 관련 단축키]

1. Command + Enter

- 결과 값만 보고자 할 때
- 해당 셀을 실행하고 커서를 해당 셀에 두는 경우

2. Shift + Enter

- 여러가지 값을 빠르게 출력할 때
- 해당 셀을 실행하고 커서를 다음 셀로 넘기는 경우

3. Alt + Enter

- 다음 작업 공간이 없을 때
- 해당 셀을 실행하고 셀을 삽입한 후 커서를 삽입한 셀로 넘기는 경우

코랩 단축키

[셀 삽입/삭제 관련 단축키]

1. Command + M D

- 셀 지우기

2. Command + M Y

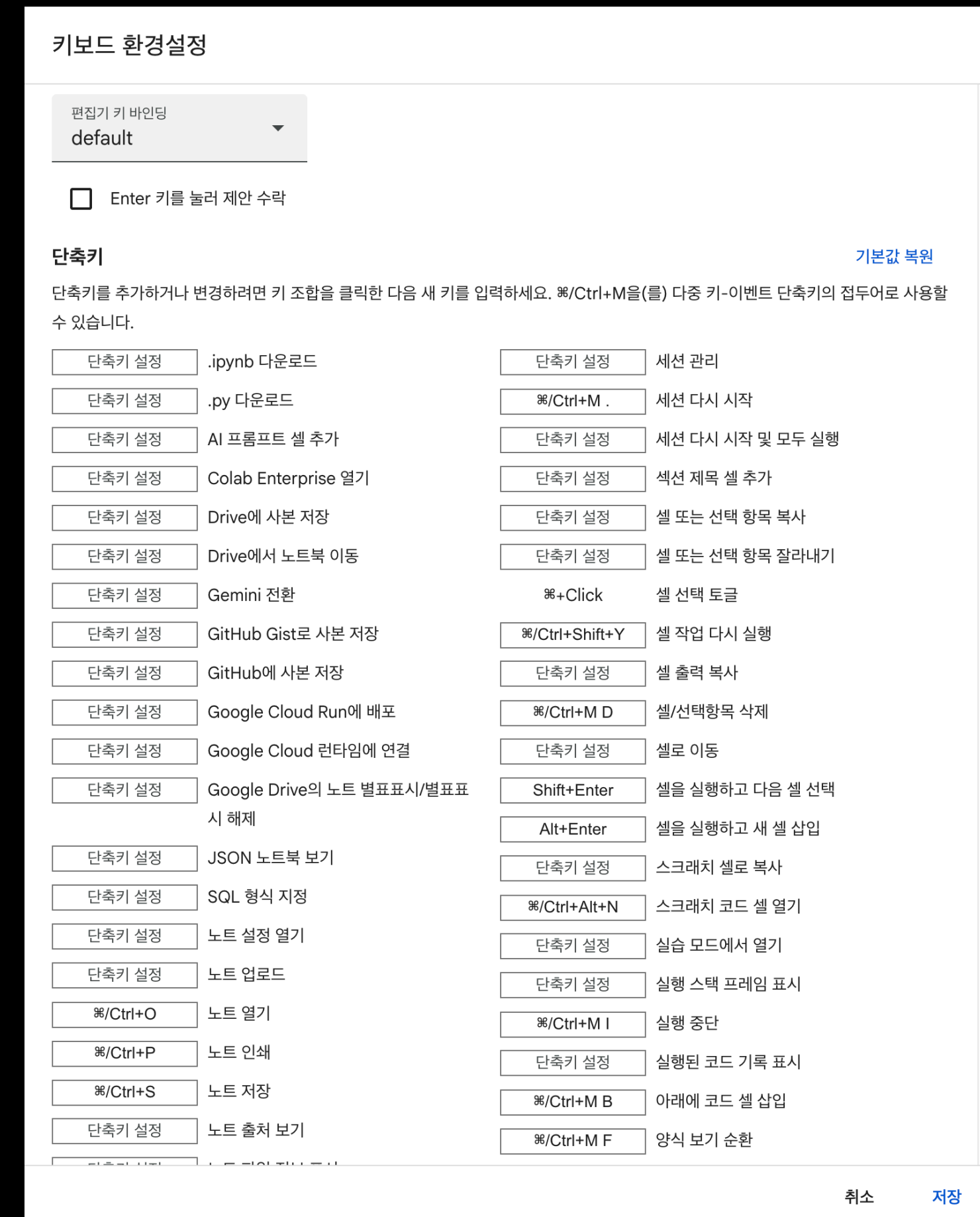
- 코드 셀로 변경

3. Command + M M

- 마크다운 셀로 변경

4. Command + M H

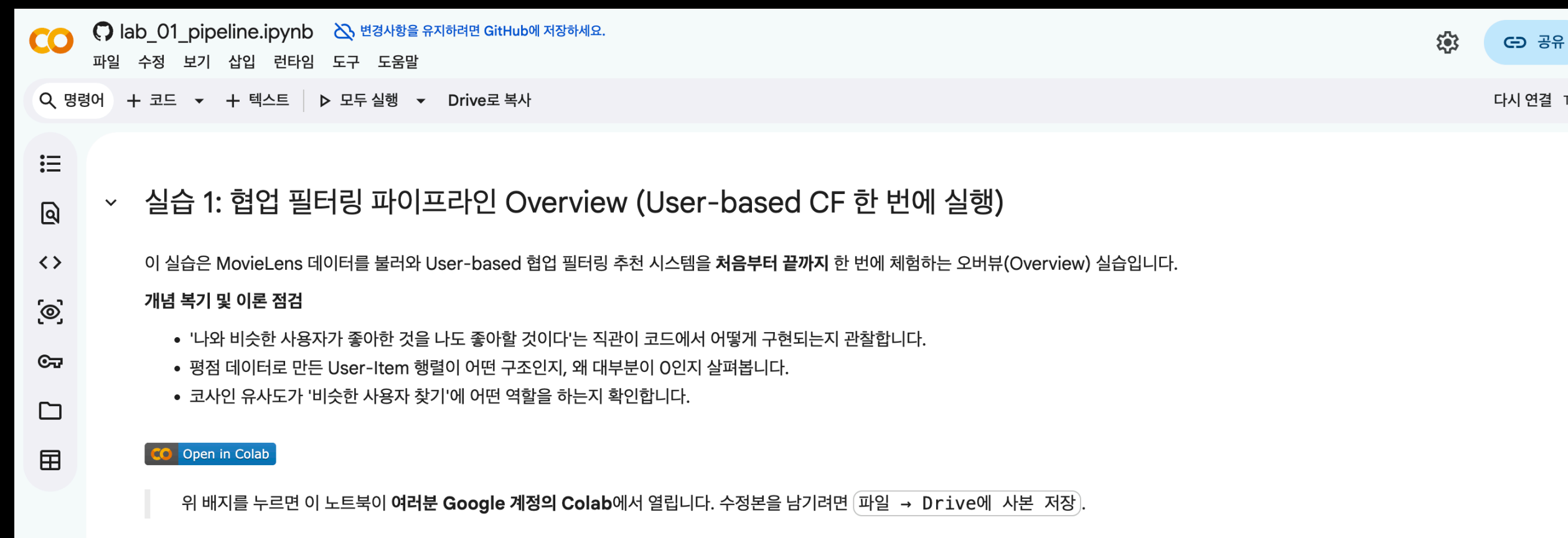
- 단축키 모음 열기



코드 실습

코드 내용을 따라하며 실습

- 중간에 놓치는 경우 아래 링크에서 내용을 확인
- https://colab.research.google.com/github/Team-Anl/A-AND-I-4TH-AI-CODE-LAB/blob/main/4%EC%A3%BC%EC%B0%A8/lab_01_pipeline.ipynb



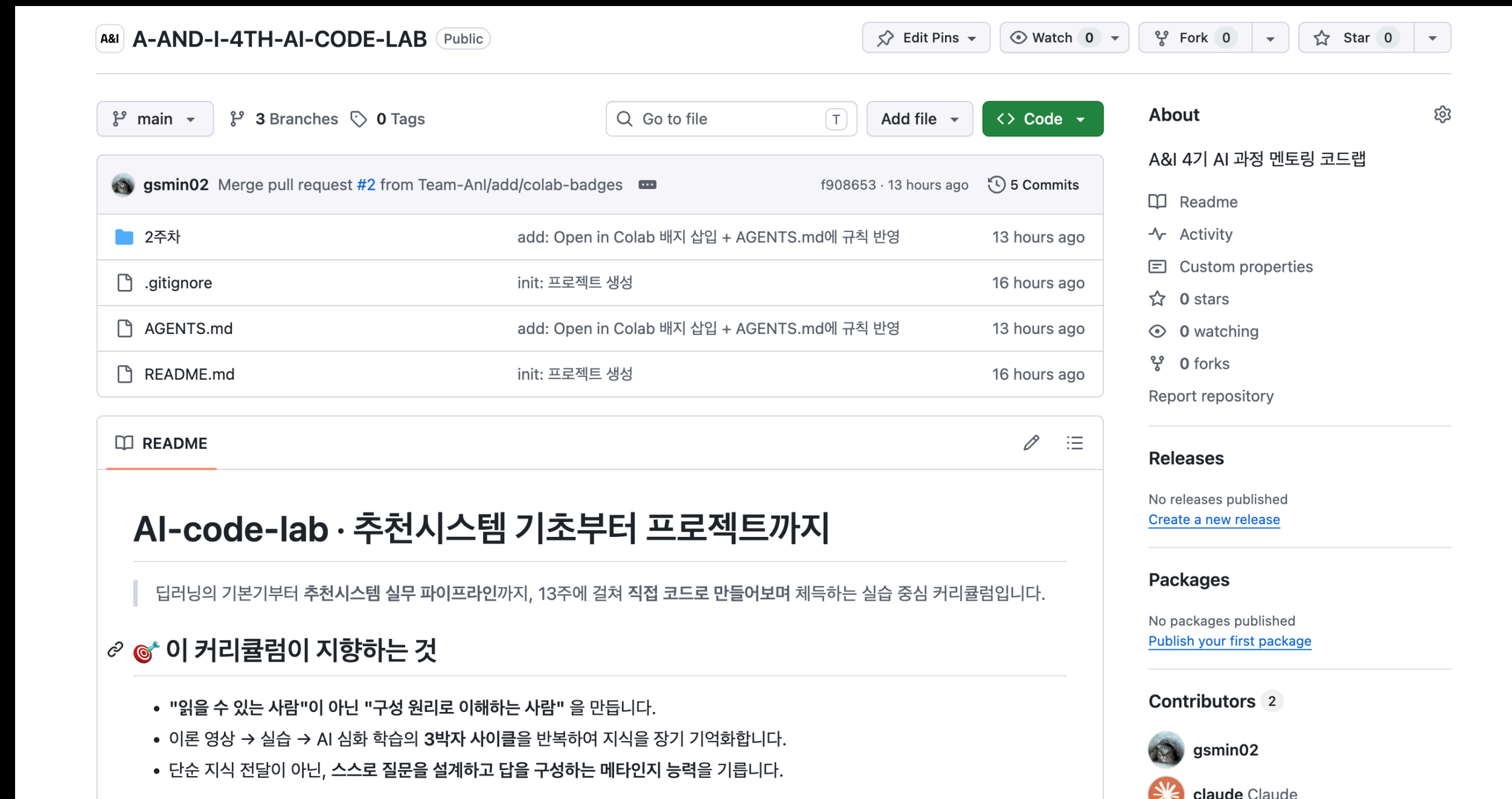
2. 코드 설명

- 작성한 코드의 세부 내용을 살펴봅니다

강의자료 확인 방법

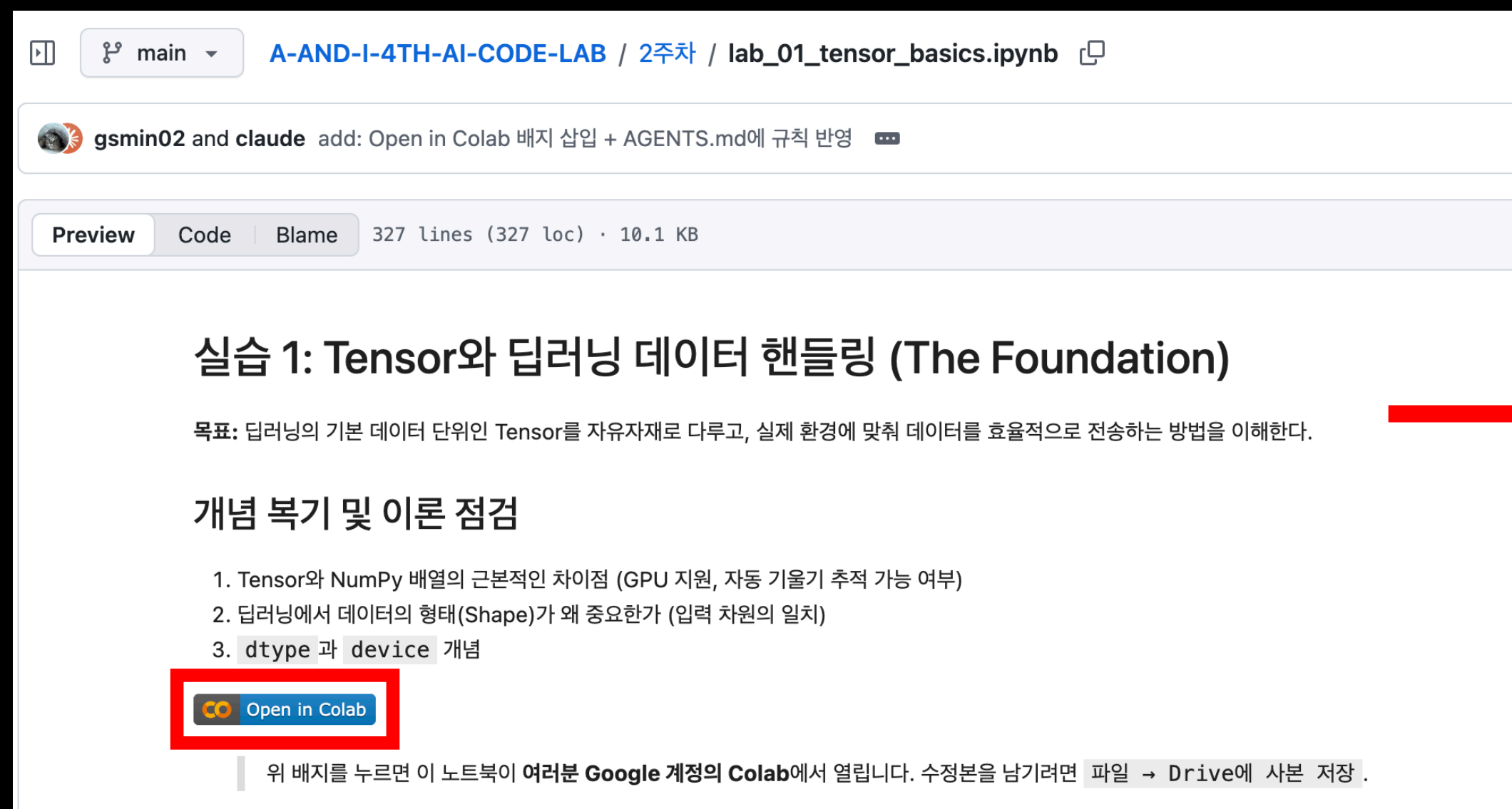
강의자료는 아래 링크를 통해 확인 가능

- <https://github.com/Team-Anl/A-AND-I-4TH-AI-CODE-LAB>

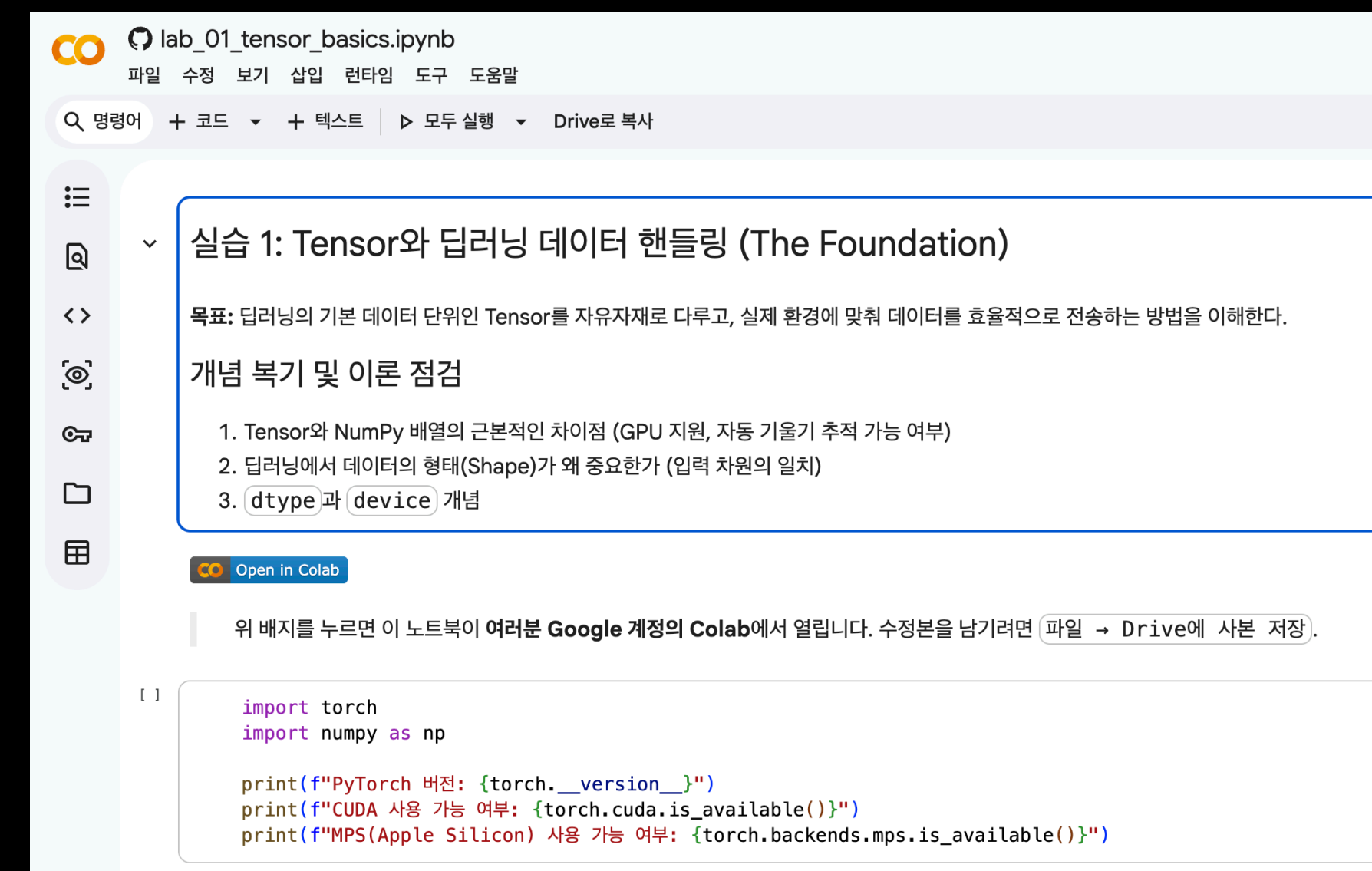


코드 실습 수행

각 실습은 제공된 ipynb 파일 상단 “Open in Colab”으로 실습 가능



The screenshot shows a GitHub repository page for the file 'lab_01_tensor_basics.ipynb'. The file is 327 lines (327 loc) and 10.1 KB. The 'Preview' tab is selected, showing the title '실습 1: Tensor와 딥러닝 데이터 핸들링 (The Foundation)'. Below the title is a description: '목표: 딥러닝의 기본 데이터 단위인 Tensor를 자유자재로 다루고, 실제 환경에 맞춰 데이터를 효율적으로 전송하는 방법을 이해한다.' followed by a section '개념 복기 및 이론 점검' with three numbered items. At the bottom, there is a red box highlighting the 'Open in Colab' button.



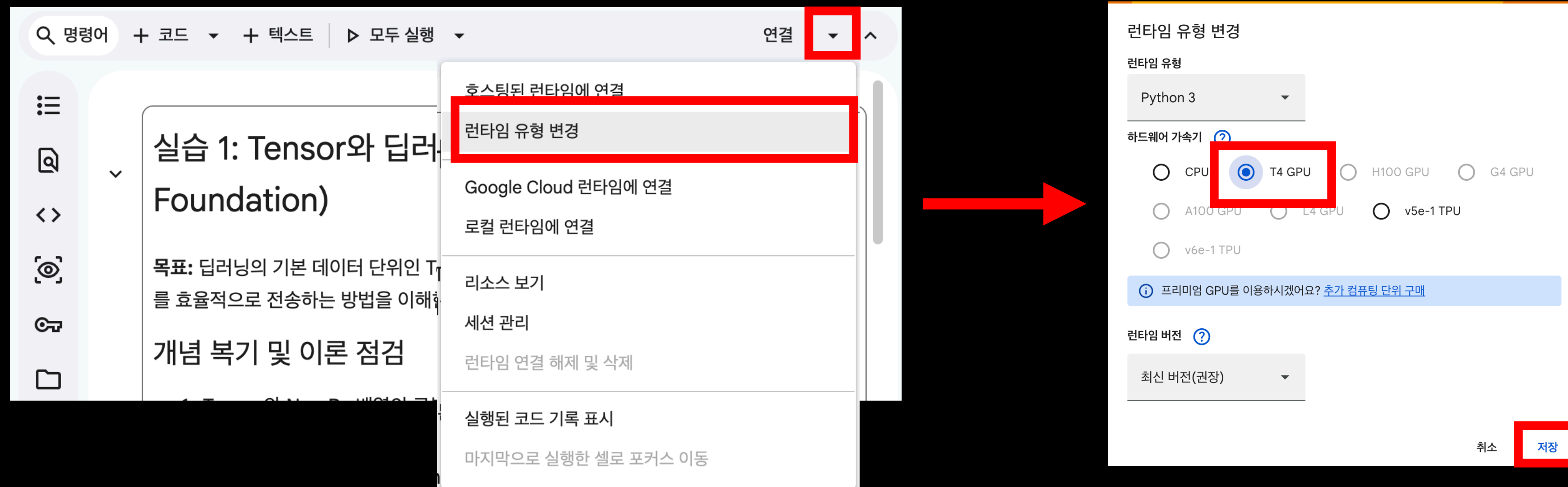
The screenshot shows the Google Colab interface for the file 'lab_01_tensor_basics.ipynb'. The title '실습 1: Tensor와 딥러닝 데이터 핸들링 (The Foundation)' is at the top. Below it is the same description as in the GitHub preview. The '개념 복기 및 이론 점검' section is also present. At the bottom, there is a code cell with the following code:

```
import torch
import numpy as np

print(f"PyTorch 버전: {torch.__version__}")
print(f"CUDA 사용 가능 여부: {torch.cuda.is_available()}")
print(f"MPS(Apple Silicon) 사용 가능 여부: {torch.backends.mps.is_available()}")
```

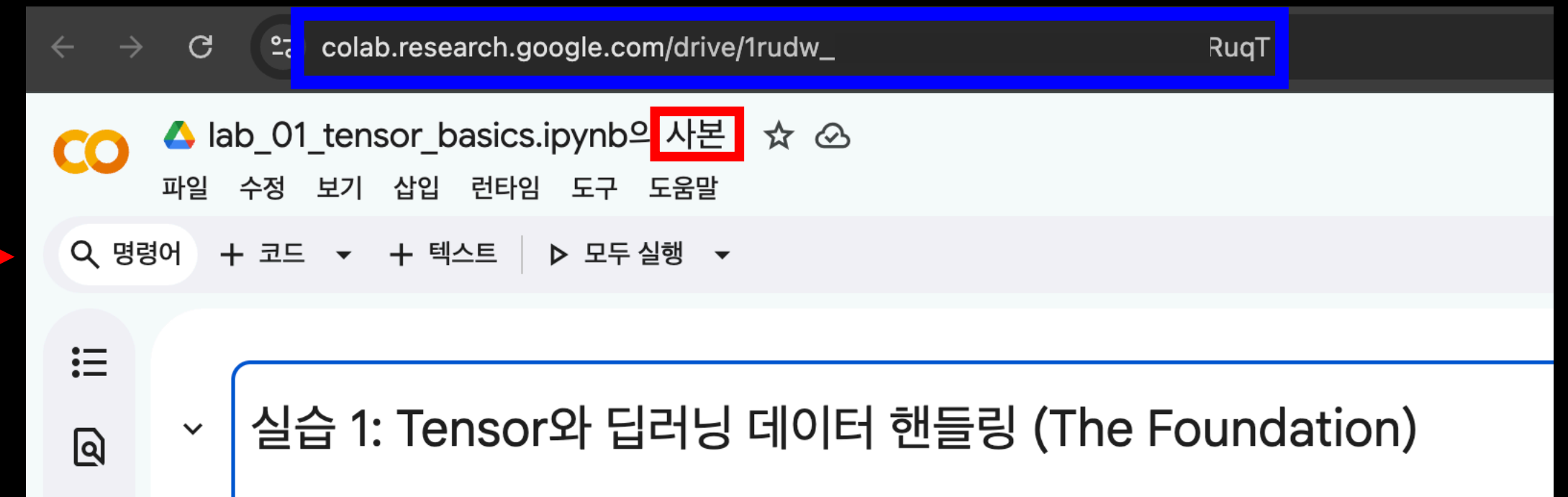
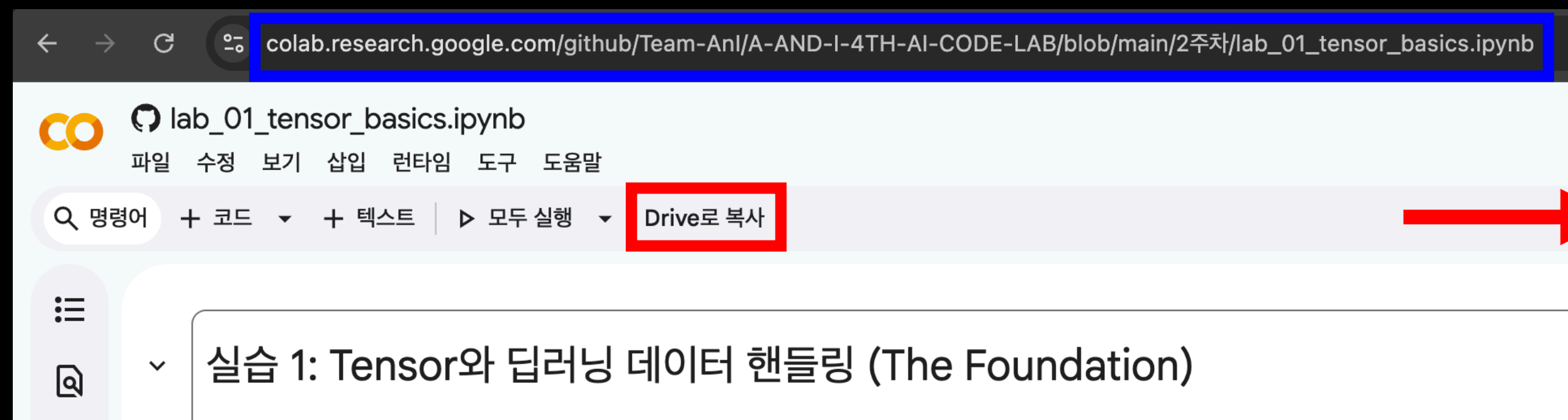

코드 실습 주의사항

실행 전 “런타임 유형 변경”을 통해 “T4 GPU”로 변경해야 함



내용 편집

내용을 변경하고 싶은 경우 “**Drive로 복사**”를 통해 편집 가능



1. 환경 준비: 라이브러리 импорт

```
import pandas as pd
import torch

torch.manual_seed(42)
```

코드 해설

- `pandas` : 테이블 형태의 데이터를 다루는 라이브러리입니다. CSV 파일 로딩과 딕셔너리 변환에 사용합니다.
- `torch` : 텐서 연산과 행렬 곱 기반 유사도 계산에 사용합니다.
- `torch.manual_seed(42)` : 난수 시드를 고정해 실험의 재현성을 보장합니다.

2. 데이터 다운로드

```
!wget -q https://files.grouplens.org/datasets/movielens/ml-100k.zip -O ml-100k.zip
!unzip -q -o ml-100k.zip
print("다운로드 완료")
```

코드 해설

- **MovieLens ml-100k**: GroupLens 연구소에서 수집한 공개 영화 평점 데이터셋입니다. 943명의 사용자가 1682편의 영화에 매긴 100,000개의 평점을 담고 있습니다.
- **u.data**: 탭(\t)으로 구분된 `user_id | item_id | rating | timestamp` 형식의 파일입니다.
- **u.item**: 영화 ID와 영화 제목 등 메타데이터를 담은 파일입니다. 추천 결과를 사람이 읽을 수 있는 형태로 출력할 때 사용합니다.

3. User-Item 평점 행렬 구성

```
df = pd.read_csv(
    'ml-100k/u.data', sep='\t', header=None,
    names=['user_id', 'item_id', 'rating', 'timestamp']
)
movies = pd.read_csv(
    'ml-100k/u.item', sep='|', header=None, encoding='latin-1',
    usecols=[0, 1], names=['item_id', 'title']
)
movie_names = dict(zip(movies['item_id'], movies['title']))

n_users = df['user_id'].max()
n_items = df['item_id'].max()

ratings = torch.zeros(n_users, n_items)
for row in df.itertuples():
    ratings[row.user_id - 1, row.item_id - 1] = row.rating

print(f"행렬 크기: {ratings.shape} (사용자 × 영화)")
```

코드 해설

- `pd.read_csv('ml-100k/u.data', sep='\t', header=None, ...)` : 평점 파일을 불러옵니다. `sep='\t'` 는 탭 구분자, `header=None` 은 첫 줄에 헤더가 없음을 의미합니다.
- `dict(zip(movies['item_id'], movies['title']))` : 영화 ID를 키, 제목을 값으로 하는 딕셔너리를 만듭니다. 추천 결과 출력 시 ID 대신 영화 제목을 표시하기 위해 사용합니다.
- `torch.zeros(n_users, n_items)` : 943×1682 크기의 0으로 채워진 행렬을 만듭니다. 0은 '아직 평가하지 않음'을 의미합니다.
- `ratings[row.user_id - 1, row.item_id - 1] = row.rating` : ml-100k의 ID는 1부터 시작하므로 `-1` 을 빼서 0-indexed 텐서 인덱스로 맞춥니다.
- 결과 행렬의 각 행은 한 사용자의 평점 벡터, 각 열은 한 영화에 대한 모든 사용자의 평점 벡터입니다. 대부분의 칸이 0인 이 구조를 **희소 행렬(Sparse Matrix)** 이라고 합니다.

4. 코사인 유사도 계산

```
norms = torch.norm(ratings, dim=1, keepdim=True).clamp(min=1e-8)
normalized = ratings / norms
user_sim = torch.mm(normalized, normalized.T) # (943, 943)

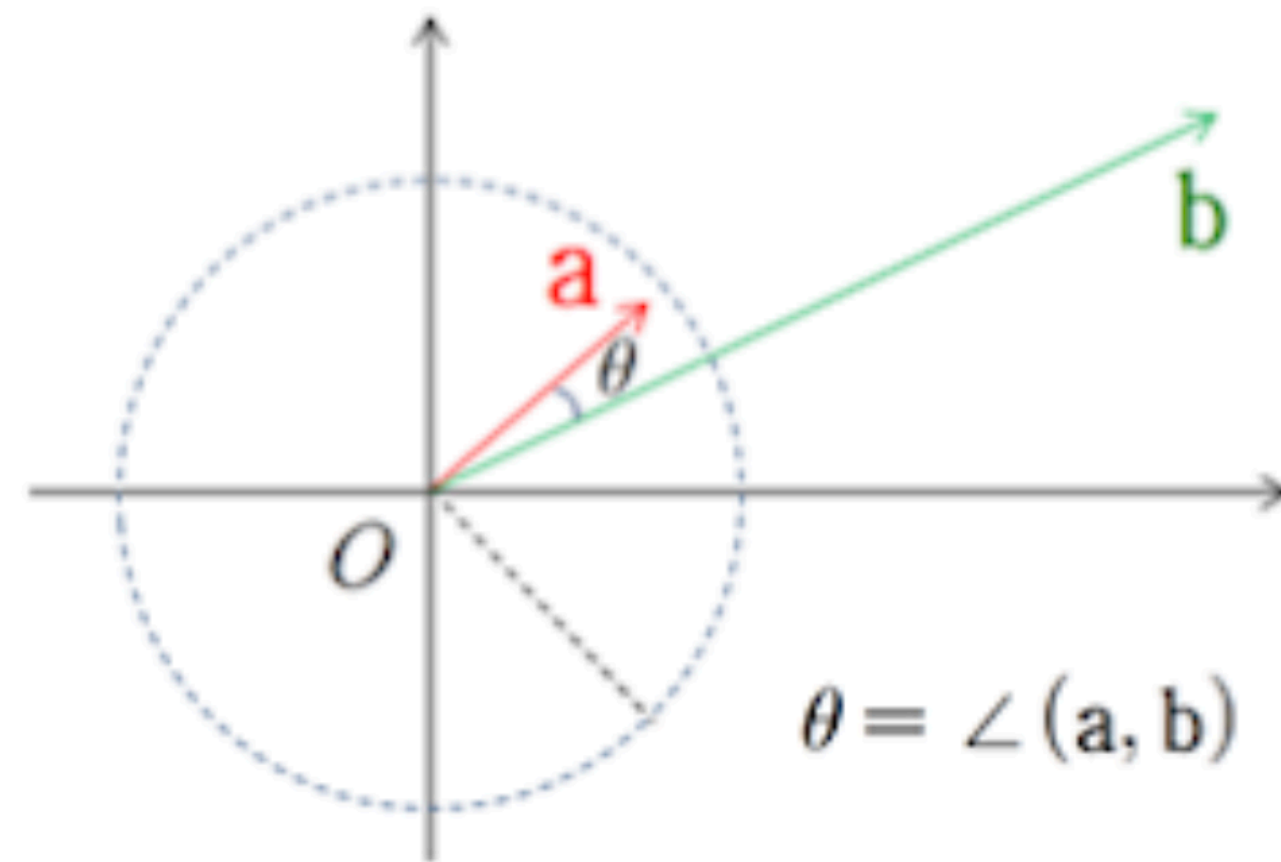
print(f"사용자 유사도 행렬 크기: {user_sim.shape}")
print(f"사용자 0과 사용자 1의 유사도: {user_sim[0, 1].item():.4f}")
```

코드 해설

코사인 유사도 공식은 $\cos(a, b) = (a \cdot b) / (\|a\| \times \|b\|)$ 입니다. 이를 두 단계로 계산합니다.

- **`torch.norm(ratings, dim=1, keepdim=True)`** : 각 사용자 평점 벡터의 크기(L2 norm)를 계산합니다. `dim=1` 은 행 방향, 즉 각 사용자별로 계산하라는 의미입니다. `keepdim=True` 는 결과 형태를 `(943, 1)` 로 유지해 이후 나눗셈에서 브로드캐스팅이 올바르게 동작하도록 합니다.
- **`.clamp(min=1e-8)`** : 한 번도 평점을 매기지 않은 사용자의 벡터 크기가 0이 되어 나눗셈 오류가 나는 것을 방지합니다.
- **`ratings / norms`** : 각 벡터를 자신의 크기로 나누면 방향만 남고 크기가 1인 단위 벡터가 됩니다. 이를 정규화(Normalization)라고 합니다.
- **`torch.mm(normalized, normalized.T)`** : 단위 벡터끼리의 내적(dot product)은 곧 코사인 유사도입니다. 행렬 곱 `A @ A^T` 는 `A` 의 모든 행 쌍의 내적을 한 번에 계산하므로, `result[i][j]` 가 사용자 i와 j의 코사인 유사도가 됩니다. 반복문 없이 한 번의 연산으로 처리합니다.

코사인 유사도(cosine similarity)



코사인 값이 크면, 사잇각은 작아지고,
유사도는 높아진다.

$$\cos \theta = \left(\frac{a}{\|a\|} \right) \cdot \left(\frac{b}{\|b\|} \right)$$

$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i \times B_i}{\sqrt{\sum_{i=1}^n (A_i)^2} \times \sqrt{\sum_{i=1}^n (B_i)^2}}$$

5. User-based CF

```
def recommend_user_based(user_idx, ratings, user_sim, top_n_neighbors=20, top_n_items=10):
    sim_scores = user_sim[user_idx].clone()
    sim_scores[user_idx] = -1

    top_neighbors = torch.topk(sim_scores, top_n_neighbors).indices

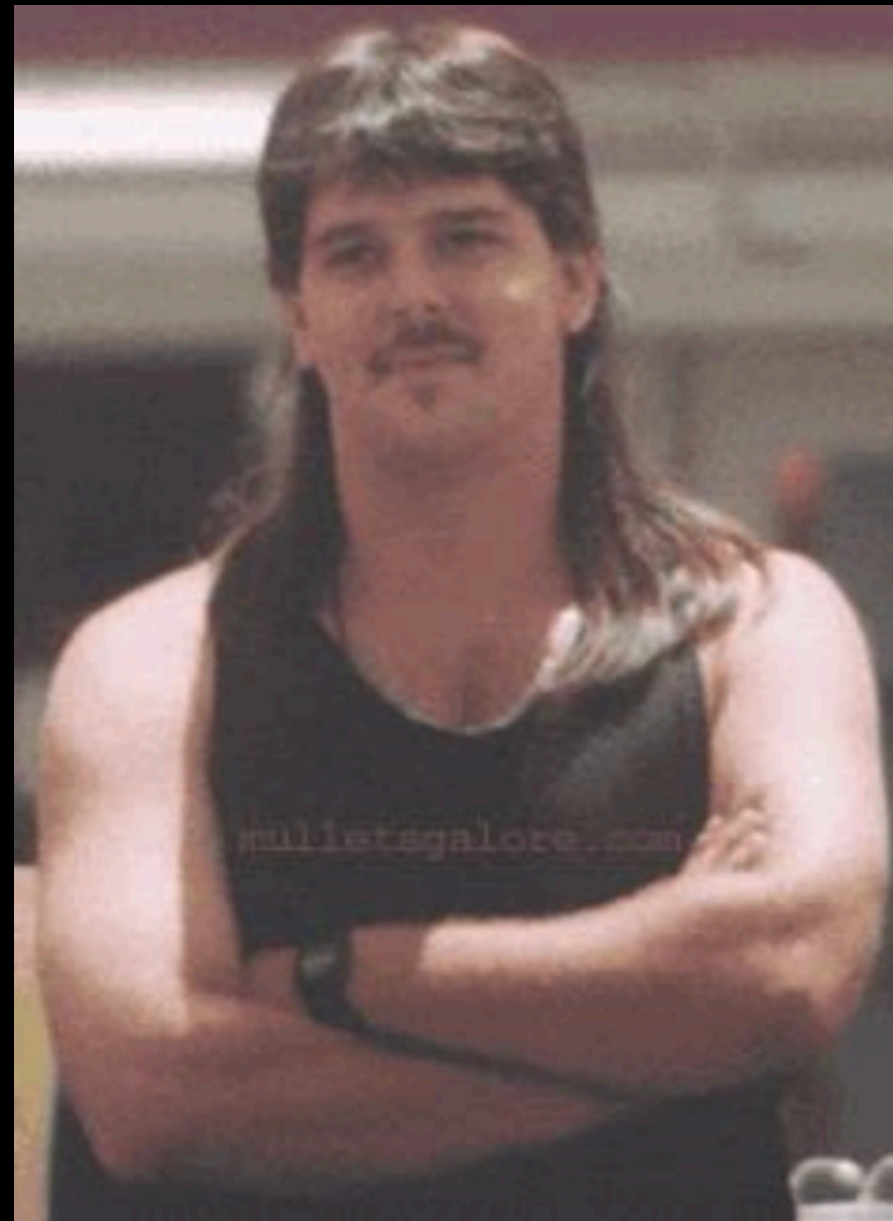
    unrated_mask = (ratings[user_idx] == 0)
    neighbor_ratings = ratings[top_neighbors]          # (top_n_neighbors, n_items)
    weights = sim_scores[top_neighbors].unsqueeze(1)   # (top_n_neighbors, 1)

    scores = (neighbor_ratings * weights).sum(dim=0)    # (n_items,)
    scores[~unrated_mask] = -float('inf')

    return torch.topk(scores, top_n_items).indices.tolist()
```

코드 해설

- `user_sim[user_idx].clone()` : 원본 `user_sim` 행렬을 수정하지 않기 위해 복사본을 만듭니다. PyTorch에서 텐서 슬라이싱은 뷰(view)를 반환하므로 복사 없이 수정하면 원본이 바뀝니다.
- `sim_scores[user_idx] = -1` : 자기 자신과의 유사도는 항상 1(최댓값)이므로, -1로 설정해 이웃 선정에서 제외합니다.
- `torch.topk(sim_scores, top_n_neighbors).indices` : 유사도 값이 가장 높은 상위 k명의 인덱스를 반환합니다. 이 k명이 이웃(neighbor)입니다. k는 하이퍼파라미터로, 너무 적으면 추천이 편향되고 너무 많으면 유사도가 낮은 사용자까지 포함됩니다.
- `neighbor_ratings * weights` : 유사도가 높은 이웃의 평점에 더 높은 가중치를 곱합니다. `weights`의 형태가 `(k, 1)`이므로 `neighbor_ratings` `(k, n_items)`와 브로드캐스팅으로 곱해집니다.
- `.sum(dim=0)` : 모든 이웃의 가중 평점을 영화별로 합산합니다. `dim=0`은 행 방향으로 합산한다는 의미입니다.
- `scores[~unrated_mask] = -float('inf')` : 이미 본 영화의 점수를 음의 무한대로 설정해 `topk` 결과에 절대 등장하지 않도록 합니다.



?

`sim_scores[user_idx] = -1` : 자기 자신과의 유사도는 항상 1(최댓값)이므로, -1로 설정해 이웃 선정에서 제외합니다.

“**나**와 가장 취향이 비슷한 사람이 누구인가?”
-> 바로 “**나**”

자신과 동일하면 아무 의미가 없음

그래서 자신을 -1로 설정하여 제외함

`torch.topk(sim_scores, top_n_neighbors).indices` : 유사도 값이 가장 높은 상위 k명의 인덱스를 반환합니다. 이 k명이 이웃(neighbor)입니다. k는 하이퍼파라미터로, 너무 적으면 추천이 편향되고 너무 많으면 유사도가 낮은 사용자까지 포함됩니다.

“맛집을 k명에게 추천 받는다면?”

k=3) 나와 가장 비슷한 3명에게 추천 받음

-> 신뢰도 높음

k=100) 100명이라 취향이 나와 다른 사람도 섞임

-> 신뢰도 낮음

`neighbor_ratings * weights` : 유사도가 높은 이웃의 평점에 더 높은 가중치를 곱합니다. `weights` 의 형태가 `(k, 1)` 이므로 `neighbor_ratings` `(k, n_items)` 와 브로드캐스팅으로 곱해집니다.

친구A) 나와 입맛이 매우 비슷함

친구B) 나와 입맛이 조금 다름

둘 중 누구의 “강추” 메뉴를 먹어야 할까?

-> 당연히 친구A가 더 높음

유사도가 높을수록 평점을 더 크게 반영

브로드캐스팅: 연결되는 모든 것에 적용


```
scores[~unrated_mask] = -float('inf') : 이미 본 영화의 점수를 음의 무한대로 설정해 topk 결과에 절대 등장하지 않도록 합니다.
```

이미 본 영화를 다시 추천 받는 것은 의미 없음

그냥 낮추는 것은 수학적으로 제외 대상이 안됨

음의 무한대로 항상 우선순위에서 밀리도록 설정

6. 추천 생성

```
user_idx = 0
ub_rec_items = recommend_user_based(user_idx, ratings, user_sim)

print(f"사용자 {user_idx + 1}번의 User-based CF 추천 영화 Top 10:")
for rank, item_idx in enumerate(ub_rec_items, 1):
    print(f" {rank:2d}. {movie_names.get(item_idx + 1, 'Unknown')}")
```

코드 해설

- **user_idx = 0** : 0-indexed로 0번은 데이터셋의 1번 사용자입니다.
- **movie_names.get(item_idx + 1, 'Unknown')** : 텐서 인덱스는 0-based이므로 +1 을 더해 MovieLens의 1-based ID로 변환한 뒤 영화 제목을 조회합니다. 딕셔너리에 없는 ID가 들어올 경우 'Unknown'을 기본값으로 반환합니다.

💡 직접 해보기: **user_idx** 를 다른 값으로 바꾸거나, **top_n_neighbors** 를 5 또는 50으로 바꾸면 추천 결과가 어떻게 달라지는지 확인해보세요.

7. Item-based CF와의 비교

```
item_norms = torch.norm(ratings.T, dim=1, keepdim=True).clamp(min=1e-8)
item_normalized = ratings.T / item_norms
item_sim = torch.mm(item_normalized, item_normalized.T) # (1682, 1682)

def recommend_item_based(user_idx, ratings, item_sim, top_n=10):
    user_ratings = ratings[user_idx]
    unrated_mask = (user_ratings == 0)
    scores = torch.mv(item_sim, user_ratings)
    scores[~unrated_mask] = -float('inf')
    return torch.topk(scores, top_n).indices.tolist()

ib_rec_items = recommend_item_based(user_idx, ratings, item_sim)

print(f"사용자 {user_idx + 1}번의 Item-based CF 추천 영화 Top 10:")
for rank, item_idx in enumerate(ib_rec_items, 1):
    print(f" {rank:2d}. {movie_names.get(item_idx + 1, 'Unknown')}")

ub_set = set(ub_rec_items)
ib_set = set(ib_rec_items)
print(f"\n겹치는 영화 ({len(ub_set & ib_set)}편):")
for item_idx in ub_set & ib_set:
    print(f" - {movie_names.get(item_idx + 1, 'Unknown')}")
```

코드 해설

- **ratings.T** : User-based CF에서는 **ratings** 의 각 행을 사용자 벡터로 사용했습니다. Item-based CF에서는 **ratings.T** 의 각 행, 즉 원본 행렬의 각 열을 영화 벡터로 사용합니다. 코드 구조는 완전히 동일하고, 입력만 전치(transpose)한 것입니다.
- **torch.mv(item_sim, user_ratings)** : 행렬-벡터 곱입니다. **item_sim** 의 각 행 *i* 는 영화 *i* 와 모든 영화의 유사도 벡터입니다. 이 벡터와 **user_ratings** (사용자의 평점 벡터)의 내적은 '영화 *i* 와 사용자가 높게 평가한 영화들의 유사도 가중합'이 됩니다. 즉, **scores[i]** 가 높을수록 사용자가 좋아할 만한 영화입니다.

3. EDA, Item vs User

- 구현 방식에 따른 모델의 결과 확인하기

Recommender Systems

Content Filtering

Collaborative Filtering

Neighborhood Methods

Latent Factor Methods:
- **Matrix Factorization**

- ...

Q & A

End of Document